

IGCSE Computer Science

Comprehensive Study Notes

Programming Basics · Algorithms · Computational Thinking

Topics Covered	Level	Syllabus
3 Core Chapters	IGCSE 0478 / 0984	Cambridge 2023–2025

Prepared by Apex STEM Tutors | 20 Years of IGCSE Excellence

Table of Contents

Chapter 1 Computational Thinking

- 1.1 What is Computational Thinking?
- 1.2 Decomposition
- 1.3 Pattern Recognition
- 1.4 Abstraction
- 1.5 Algorithm Design
- 1.6 Examinable Examples
- 1.7 Practice Exercises

Chapter 2 Algorithms

- 2.1 What is an Algorithm?
- 2.2 Pseudocode Conventions
- 2.3 Flowcharts
- 2.4 Sequence, Selection, Iteration
- 2.5 Searching Algorithms – Linear & Binary
- 2.6 Sorting Algorithms – Bubble & Merge
- 2.7 Trace Tables
- 2.8 Examinable Examples
- 2.9 Practice Exercises

Chapter 3 Basics of Programming

- 3.1 Variables, Constants & Data Types
- 3.2 Input / Output
- 3.3 Arithmetic & Comparison Operators
- 3.4 Selection – IF / CASE
- 3.5 Iteration – FOR / WHILE / REPEAT
- 3.6 Arrays
- 3.7 Procedures & Functions
- 3.8 String Handling
- 3.9 File Handling
- 3.10 Examinable Examples
- 3.11 Practice Exercises

Chapter 1

Computational Thinking

Computational thinking is the foundation skill behind all computer science. Cambridge IGCSE 0478 / 0984 tests these four pillars in Section A of both Papers 1 and 2, and they underpin every programming question.

1.1 What is Computational Thinking?

Computational thinking is a **problem-solving approach** that uses techniques drawn from computer science to formulate problems and express their solutions in a way that a computer (or human) can carry out. It is *not* about programming — it is a **mental framework**.

Definition: Computational Thinking

A set of problem-solving skills including decomposition, pattern recognition, abstraction and algorithm design that enable complex problems to be solved systematically.

Pillar	Definition	Real-World Example
Decomposition	Breaking a complex problem into smaller, manageable sub-problems	Building a website: design, development, content, testing
Pattern Recognition	Identifying similarities, trends or regularities within a problem	Designing multiple websites — reuse same solution
Abstraction	Removing unnecessary detail and focusing on what matters	Designing a search engine: focus on what matters
Algorithm Design	Developing step-by-step instructions to solve a problem	Sorting a list of names alphabetically

1.2 Decomposition

Decomposition means breaking a large problem into smaller parts that are easier to understand, solve and test. Each sub-problem can be solved independently and the solutions combined.

Worked Example 1.1 – Decompose a 'School Timetable System'

Main Problem: Create a school timetable system

Sub-problem 1: Store teacher availability

Sub-problem 2: Store classroom capacity

Sub-problem 3: Assign subjects to rooms and times

Sub-problem 4: Check for conflicts

Sub-problem 5: Print timetable for each class

1.3 Pattern Recognition

Pattern recognition involves noticing when a new problem is **similar to one already solved**. By identifying patterns, programmers can reuse existing solutions — reducing development time and errors.

Examples of Pattern Recognition in Computing

All shapes (circle, square, triangle) have an area — use the same function structure for each.

Every social media profile has: name, photo, bio, posts — reuse the same template.

Finding the maximum of a list uses the same scan-and-compare logic regardless of data type.

1.4 Abstraction

Abstraction is the process of **hiding unnecessary detail** while keeping relevant information. In programming, abstraction lets us work with complex systems without understanding every internal detail.

Level of Abstraction	What We See	What is Hidden
Driving a car	Steering wheel, pedals	Engine combustion, gear mechanisms
Using a search engine	Search bar, results	Indexing algorithms, server farms
Calling a function	Function name and parameters	Internal implementation
Using a map	Roads, landmarks	Individual buildings, terrain details

1.5 Algorithm Design

Once a problem is decomposed and abstracted, a precise **step-by-step solution (algorithm)** must be designed. Good algorithms are correct, efficient, unambiguous, and finite.

- **Input:** Takes zero or more inputs from the user or a data source
- **Output:** Produces at least one result/output
- **Definiteness:** Each step is clear and unambiguous
- **Finiteness:** Terminates after a finite number of steps
- **Effectiveness:** Each step is simple enough to be carried out

1.6 Examinable Examples & Worked Questions

Q1 [2 marks] A student wants to build an app that recommends movies. State two sub-problems they could identify using decomposition.

Answer: Any two of: store list of movies / store user preferences / match preferences to movies / display recommendations / handle user ratings.

Q2 [2 marks] Explain what is meant by abstraction and give one example from everyday computing.

Answer: Abstraction means removing unnecessary detail and keeping only what is relevant [1].

Example: using a printer — the user clicks 'Print' without knowing how the printer formats data into dots on a page [1].

Q3 [1 mark] A programmer notices that calculating the area of a rectangle and the area of a triangle both require multiplying two values. Which computational thinking technique is being applied?

Answer: Pattern recognition — the same calculation pattern (multiplication of two values) is identified across both problems.

1.7 Practice Exercises

Exercise 1.1: Use decomposition to break down the problem: 'Create an online shopping cart'. List at least four sub-problems.

Exercise 1.2: Explain how pattern recognition would help a developer building both a 'student registration form' and a 'patient registration form'.

Exercise 1.3: A weather app shows temperature, wind speed and a weather icon. Using abstraction, explain what information is hidden from the user.

Exercise 1.4: State four properties that all good algorithms must have.

Exercise 1.5: Describe the difference between abstraction and decomposition. Give a different example for each.

Examiner's Tip

In the exam, when asked to 'identify the computational thinking technique', always name it AND briefly justify — examiners award the second mark for the justification, not just the label.

Chapter 2

Algorithms

Algorithms are the heart of computing. In IGCSE Paper 2 (Problem Solving), you will write pseudocode, trace algorithms, draw flowcharts, and evaluate efficiency. Mastery here directly translates to marks in Section B.

2.1 What is an Algorithm?

Definition: Algorithm

A finite, ordered set of unambiguous, step-by-step instructions that solves a problem or achieves a defined goal in a finite amount of time.

Algorithms can be represented in three main ways in IGCSE: **pseudocode**, **flowcharts**, and **program code**.

2.2 Pseudocode Conventions (Cambridge IGCSE)

Cambridge uses its own pseudocode standard. You **must** follow these conventions in Paper 2 — not Python/Java syntax.

Construct	Cambridge Pseudocode	Notes
Assignment	<code>x <-- 10</code>	Use <-- not = or :=
Input	<code>INPUT name</code>	Reads from user
Output	<code>OUTPUT 'Hello', name</code>	Prints text / value
IF block	<code>IF x > 5 THEN ... ELSE ... ENDIF</code>	ENDIF required
CASE	<code>CASE OF x ... ENDCASE</code>	For multiple conditions
FOR loop	<code>FOR i <- 1 TO 10 ... NEXT i</code>	Counted loop
WHILE loop	<code>WHILE x <> 0 DO ... ENDWHILE</code>	Pre-condition
REPEAT loop	<code>REPEAT ... UNTIL x = 0</code>	Post-condition
Procedure	<code>PROCEDURE name(param) ... ENDPROCEDURE</code>	Procedure
Function	<code>FUNCTION name(param) RETURNS type ... ENDFUNCTION</code>	Function

Examiner's Tip

Cambridge pseudocode is case-sensitive with keywords. Always write keywords in CAPITALS (IF, THEN, ELSE, ENDIF, FOR, WHILE, etc.). Losing marks for wrong syntax in pseudocode is very common — practise the exact style!

2.3 Flowcharts – Symbols

Symbol Shape	Name	Used For
Rounded rectangle (oval)	Terminator	START / STOP of algorithm
Rectangle	Process	Calculations, assignments, operations
Parallelogram	Input / Output	INPUT or OUTPUT of data
Diamond	Decision	Yes/No question — branches the flow
Arrow	Flow line	Shows direction / sequence of steps

Common Mistake: Flowchart Mistakes

A decision diamond MUST have exactly two exits labelled YES and NO (or True/False). Always include START and STOP terminators. Never draw arrows that go nowhere.

2.4 Sequence, Selection and Iteration

Sequence

Statements executed one after another in order.

PSEUDOCODE

```
OUTPUT 'Enter number'
INPUT number
result <-- number * 2
OUTPUT result
```

Selection

A decision is made; different code runs depending on condition.

PSEUDOCODE

```
IF score >= 50 THEN
    OUTPUT 'Pass'
ELSE
    OUTPUT 'Fail'
ENDIF
```

Iteration

A block of code repeats — either a fixed number of times (FOR) or

PSEUDOCODE

```
FOR i <- 1 TO 5
    OUTPUT i * i
NEXT i
```

2.5 Searching Algorithms

Linear Search

Checks each element in sequence from the first until the target is found or the list is exhausted. Works on *unsorted* or sorted data.

PSEUDOCODE

```
PROCEDURE LinearSearch(list[], target)
  found <-- FALSE
  index <-- 0
  WHILE index < LENGTH(list) AND found = FALSE DO
    IF list[index] = target THEN
      found <-- TRUE
    ELSE
      index <-- index + 1
    ENDIF
  ENDWHILE
  IF found = TRUE THEN
    OUTPUT 'Found at index ', index
  ELSE
    OUTPUT 'Not found'
  ENDIF
ENDPROCEDURE
```

Binary Search

Repeatedly halves the search space. Requires a **sorted** list. Much more efficient than linear search for large datasets.

PSEUDOCODE

```

PROCEDURE BinarySearch(list[], target)
  low <-- 0
  high <-- LENGTH(list) - 1
  found <-- FALSE
  WHILE low <= high AND found = FALSE DO
    mid <-- (low + high) DIV 2
    IF list[mid] = target THEN
      found <-- TRUE
      OUTPUT 'Found at index ', mid
    ELSE IF target < list[mid] THEN
      high <-- mid - 1
    ELSE
      low <-- mid + 1
    ENDIF
  ENDWHILE
  IF found = FALSE THEN OUTPUT 'Not found' ENDIF
ENDPROCEDURE

```

Feature	Linear Search	Binary Search
Pre-condition	List can be unsorted	List MUST be sorted
Best case	$O(1)$ — first element	$O(1)$ — middle element
Worst case	$O(n)$ — checks all n items	$O(\log n)$ — halves each time
Suitable for	Small / unsorted lists	Large sorted lists
Example ($n=1000$)	Up to 1000 comparisons	Up to 10 comparisons

2.6 Sorting Algorithms

Bubble Sort

Repeatedly compares adjacent elements and swaps them if in the wrong order. After each pass, the largest unsorted element 'bubbles' to its correct position.

PSEUDOCODE

```

PROCEDURE BubbleSort(list[])
  n <-- LENGTH(list)
  FOR pass <- 1 TO n - 1
    FOR i <- 0 TO n - pass - 1
      IF list[i] > list[i+1] THEN
        temp <-- list[i]
        list[i] <-- list[i+1]
        list[i+1] <-- temp
      ENDIF
    NEXT i
  NEXT pass
  OUTPUT list
ENDPROCEDURE

```

Worked Example 2.1 – Bubble Sort Trace (list = [5, 3, 8, 1])

Pass 1: Compare 5,3 → swap → [3,5,8,1]

Compare 5,8 → no swap → [3,5,8,1]

Compare 8,1 → swap → [3,5,1,8] (8 is now in place)

Pass 2: Compare 3,5 → no swap; Compare 5,1 → swap → [3,1,5,8]

Pass 3: Compare 3,1 → swap → [1,3,5,8]

Sorted list: [1, 3, 5, 8] (3 passes needed for 4 elements)

2.7 Trace Tables

A trace table is used to manually track the value of variables as an algorithm executes. It is a very common IGCSE exam question — usually worth 4–6 marks.

Worked Example 2.2 – Trace Table

Algorithm: $x \leftarrow 1$; WHILE $x < 5$ DO; $x \leftarrow x * 2 + 1$; OUTPUT x ; ENDWHILE

Trace table tracks: x , output

Start: $x = 1$

Iteration 1: $x = 1 * 2 + 1 = 3 \rightarrow$ OUTPUT 3

Iteration 2: $x = 3 * 2 + 1 = 7 \rightarrow 7 \geq 5$, loop exits

Final output: 3

Iteration	x (before)	Condition $x < 5$	x (after $x*2+1$)	OUTPUT
Start	1	TRUE	3	
1	3	TRUE	7	3

2	7	FALSE	—	
---	---	-------	---	--

2.8 Examinable Examples & Worked Questions

Q1 [4 marks] A sorted list contains [2, 7, 11, 15, 20, 28, 33, 42]. Show the steps of a binary search for the value 20, clearly stating low, mid and high at each step.

Answer: Step 1: low=0, high=7, mid=3 → list[3]=15 < 20 → low=4 Step 2: low=4, high=7, mid=5 → list[5]=28 > 20 → high=4 Step 3: low=4, high=4, mid=4 → list[4]=20 = 20 → FOUND at index 4 [1 mark per correct step]

Q2 [3 marks] State three differences between a linear search and a binary search.

Answer: 1. Binary search requires a sorted list; linear does not. 2. Binary search is more efficient for large lists ($O \log n$ vs $O n$). 3. Linear search checks elements one by one; binary halves the search space each time.

Q3 [4 marks] Complete a trace table for this pseudocode with input 4: x <- INPUT; total <- 0; FOR i <- 1 TO x; total <- total + i; NEXT i; OUTPUT total

Answer: i=1: total=1; i=2: total=3; i=3: total=6; i=4: total=10; OUTPUT: 10

2.9 Practice Exercises

Exercise 2.1: Write pseudocode using a WHILE loop that inputs numbers from the user until 0 is entered and outputs their total.

Exercise 2.2: Trace the bubble sort algorithm on the list [9, 4, 6, 2]. Show each pass clearly.

Exercise 2.3: A list is [10, 22, 35, 47, 58, 69, 74, 88]. Perform a binary search for 47. Show low, mid, high at each step.

Exercise 2.4: Draw a flowchart for an algorithm that reads 10 numbers and outputs how many are greater than 50.

Exercise 2.5: Write pseudocode for a procedure that takes an array of 5 integers and outputs the largest value.

Exercise 2.6: Create a trace table for: n<-5; result<-1; WHILE n>1 DO; result<-result*n; n<-n-1; ENDWHILE; OUTPUT result

Common Mistake: Trace Table Errors

Always show EVERY row — even rows where a variable doesn't change. Leave a cell blank rather than writing 'same'. Show each iteration on its own row.

Chapter 3

Basics of Programming

This chapter covers all the core programming constructs tested in IGCSE Paper 2 (Problem Solving & Programming). Every concept has pseudocode examples and exam-style questions.

3.1 Variables, Constants and Data Types

A **variable** is a named memory location whose value can change during execution. A **constant** is a named value that does *not* change.

PSEUDOCODE

```
// Variable declaration and assignment
name <-- 'Alice'
age <-- 17
average <-- 85.5
passed <-- TRUE

// Constant declaration
CONSTANT PI <-- 3.14159
CONSTANT MAX_SIZE <-- 100
```

Data Type	Description	Example Values	IGCSE Keyword
Integer	Whole numbers (positive, negative, zero)	0, 5, -12, 1000	INTEGER
Real / Float	Numbers with decimal places	3.14, -0.5, 99.9	REAL
Char	A single character	'A', '7', '!	CHAR
String	A sequence of characters	'Hello', 'IGCSE'	STRING
Boolean	True or false only	TRUE, FALSE	BOOLEAN

Examiner's Tip

When choosing a data type in an exam: use INTEGER for counts/ages, REAL for prices/measurements, STRING for names/codes, BOOLEAN for yes/no flags. Always justify your choice.

3.2 Input and Output

PSEUDOCODE

```

OUTPUT 'Enter your name: '
INPUT userName
OUTPUT 'Enter your age: '
INPUT userAge
OUTPUT 'Hello, ', userName, '! You are ', userAge, ' years old.'

```

3.3 Arithmetic and Comparison Operators

Category	Operator	Meaning	Example	Result
Arithmetic	+	Addition	5 + 3	8
Arithmetic	-	Subtraction	10 - 4	6
Arithmetic	*	Multiplication	6 * 7	42
Arithmetic	/	Division (real result)	7 / 2	3.5
Arithmetic	DIV	Integer division	7 DIV 2	3
Arithmetic	MOD	Remainder	7 MOD 2	1
Arithmetic	^	Power / Exponent	2 ^ 8	256
Comparison	=	Equal to	x = 5	TRUE/FALSE
Comparison	<>	Not equal to	x <> 5	TRUE/FALSE
Comparison	< <=	Less than / Less than or equal to	x < 10	TRUE/FALSE
Comparison	> >=	Greater than / Greater than or equal to	x > 0	TRUE/FALSE
Logical	AND	Both conditions true	x>0 AND x<10	TRUE/FALSE
Logical	OR	At least one true	x=1 OR x=5	TRUE/FALSE
Logical	NOT	Negates condition	NOT(x=0)	TRUE/FALSE

3.4 Selection – IF and CASE

IF / ELSE IF / ELSE

PSEUDOCODE

```
INPUT score
IF score >= 90 THEN
    OUTPUT 'Grade A'
ELSE IF score >= 75 THEN
    OUTPUT 'Grade B'
ELSE IF score >= 60 THEN
    OUTPUT 'Grade C'
ELSE
    OUTPUT 'Grade U'
ENDIF
```

CASE OF (for fixed discrete values)**PSEUDOCODE**

```
INPUT dayNumber
CASE OF dayNumber
    1 : OUTPUT 'Monday'
    2 : OUTPUT 'Tuesday'
    3 : OUTPUT 'Wednesday'
    4 : OUTPUT 'Thursday'
    5 : OUTPUT 'Friday'
    OTHERWISE : OUTPUT 'Weekend'
ENDCASE
```

3.5 Iteration – FOR, WHILE and REPEAT

FOR loop (count-controlled)**PSEUDOCODE**

```
FOR i <- 1 TO 5
    OUTPUT i * i
NEXT i
// Outputs: 1, 4, 9, 16, 25
```

WHILE loop (pre-condition — may not execute at all)

PSEUDOCODE

```
total <- 0
INPUT number
WHILE number <> -1 DO
    total <- total + number
    INPUT number
ENDWHILE
OUTPUT 'Total: ', total
```

REPEAT...UNTIL (post-condition — executes at least once)**PSEUDOCODE**

```
REPEAT
    OUTPUT 'Enter a positive number: '
    INPUT num
UNTIL num > 0
OUTPUT 'You entered: ', num
```

3.6 Arrays

An array is a **fixed-size collection of elements of the same data type**, stored under one variable name. Elements are accessed by their index (position), starting at 0 in Cambridge pseudocode.

PSEUDOCODE

```
// 1D Array — declare and populate
DECLARE scores[5] : INTEGER
scores[0] <-- 88
scores[1] <-- 72
scores[2] <-- 95
scores[3] <-- 61
scores[4] <-- 83

// Calculate average using a loop
total <-- 0
FOR i <- 0 TO 4
    total <-- total + scores[i]
NEXT i
average <-- total / 5
OUTPUT 'Average: ', average
```

2D Arrays

A 2D array stores data in rows and columns — like a spreadsheet.

```
DECLARE grid[3][3] : INTEGER // 3 rows, 3 columns
```

```
grid[0][0] <-- 1 // Row 0, Column 0
```

Access with two indices: grid[row][col]

3.7 Procedures and Functions

Both procedures and functions are **named blocks of code** (subroutines). The key difference: a **function** returns a value; a **procedure** does not.

PSEUDOCODE

```
// PROCEDURE – no return value
PROCEDURE Greet(name : STRING)
    OUTPUT 'Hello, ', name, '!'
ENDPROCEDURE

// FUNCTION – returns a value
FUNCTION Square(n : INTEGER) RETURNS INTEGER
    RETURN n * n
ENDFUNCTION

// Calling them:
CALL Greet('Alice')
result <-- Square(7)
OUTPUT result    // Outputs: 49
```

Feature	Procedure	Function
Returns a value?	No	Yes — always
Called with	CALL name(params)	name(params) in expression
RETURNS keyword?	No	Required
Use when?	Performing an action	Computing and returning a result

3.8 String Handling

Function / Operation	Syntax (Cambridge)	Example	Result
Length of string	LENGTH(str)	LENGTH('Hello')	5
Left n characters	LEFT(str, n)	LEFT('Computer',4)	'Comp'
Right n characters	RIGHT(str, n)	RIGHT('Science',3)	'nce'

Substring (mid)	MID(str, start, len)	MID('Cambridge',4,6)	'bridge'
Upper case	UCASE(str)	UCASE('igcse')	'IGCSE'
Lower case	LCASE(str)	LCASE('HELLO')	'hello'
String to integer	INT(str)	INT('42')	42
Integer to string	STR(num)	STR(42)	'42'
Concatenation	str1 & str2	'Hello' & ' World'	'Hello World'

PSEUDOCODE

```
// Password validation example using string functions
INPUT password
IF LENGTH(password) >= 8 AND LENGTH(password) <= 20 THEN
    OUTPUT 'Password accepted'
ELSE
    OUTPUT 'Password must be 8-20 characters'
ENDIF
```

3.9 File Handling

Files allow data to **persist between program runs**. Cambridge pseudocode uses OPENFILE, READFILE, WRITEFILE, and CLOSEFILE.

PSEUDOCODE

```
// Writing to a file
OPENFILE 'scores.txt' FOR WRITE
WRITEFILE 'scores.txt', 'Alice: 88'
WRITEFILE 'scores.txt', 'Bob: 75'
CLOSEFILE 'scores.txt'

// Reading from a file
OPENFILE 'scores.txt' FOR READ
WHILE NOT EOF('scores.txt') DO
    READFILE 'scores.txt', line
    OUTPUT line
ENDWHILE
CLOSEFILE 'scores.txt'
```

Mode	Keyword	Purpose
Read	FOR READ	Opens existing file to read from beginning
Write	FOR WRITE	Creates new file (or overwrites existing)
Append	FOR APPEND	Opens existing file to add data at end

3.10 Examinable Examples & Worked Questions

Q1 [6 marks] Write pseudocode for a program that: inputs 10 numbers from the user, stores them in an array, then outputs the largest number.

Answer: DECLARE nums[10] : REAL FOR i <- 0 TO 9; INPUT nums[i]; NEXT i largest <- nums[0] FOR i <- 1 TO 9; IF nums[i] > largest THEN largest <- nums[i] ENDIF; NEXT i OUTPUT 'Largest: ', largest

Q2 [3 marks] A function called CelsiusToFahrenheit takes a REAL temperature in Celsius and returns the Fahrenheit equivalent ($F = C * 9/5 + 32$). Write this function.

Answer: FUNCTION CelsiusToFahrenheit(celsius : REAL) RETURNS REAL RETURN celsius * 9 / 5 + 32 ENDFUNCTION

Q3 [4 marks] Write pseudocode that reads all lines from a file called 'names.txt' and counts how many lines contain the letter 'A'.

Answer: count <- 0 OPENFILE 'names.txt' FOR READ WHILE NOT EOF('names.txt') DO READFILE 'names.txt', line IF 'A' IN UCASE(line) THEN count <- count + 1 ENDIF ENDWHILE CLOSEFILE 'names.txt' OUTPUT 'Lines with A: ', count

3.11 Practice Exercises

Exercise 3.1: Write pseudocode that asks the user for a password up to 3 times. If correct ('Secret123'), output 'Access granted'. Otherwise output 'Access denied'.

Exercise 3.2: Write a FUNCTION called IsPrime that takes an INTEGER and returns a BOOLEAN (TRUE if prime, FALSE otherwise).

Exercise 3.3: Declare a 2D array 4x4 of INTEGERS. Write pseudocode to fill it with the product of its row and column indices (row * col).

Exercise 3.4: Write pseudocode for a menu-driven program using CASE OF with options: 1-Add, 2-Subtract, 3-Multiply, 4-Quit. Validate the input.

Exercise 3.5: Write a program that reads names from 'students.txt', stores them in an array, sorts the array using bubble sort, and writes the sorted names to 'sorted.txt'.

Exercise 3.6: Write a PROCEDURE called PrintTable that takes an INTEGER n and prints the multiplication table for n from 1 to 12.

Common Mistake: Assignment Operator

Cambridge uses <-- (arrow) for assignment, NOT = or :=. Using = instead of <-- in pseudocode loses marks. Use = only for COMPARISON (e.g. IF x = 5).

Quick-Revision Summary

Computational Thinking

4 pillars: Decomposition, Pattern Recognition, Abstraction, Algorithm Design.

Decomposition = break problem into sub-problems.

Abstraction = remove unnecessary detail, keep the essential.

Pattern recognition = reuse solutions to similar problems.

Algorithms

Algorithm = finite, unambiguous, ordered set of steps.

Linear search: $O(n)$, works unsorted. Binary search: $O(\log n)$, needs sorted list.

Bubble sort: compare and swap adjacent pairs, repeat $n-1$ passes.

Trace tables: track every variable change through every iteration.

Flowchart symbols: oval=start/stop, rectangle=process, diamond=decision, parallelogram=I/O.

Programming Basics

Variables change; constants don't. Choose data types carefully.

Cambridge assignment uses `<--` (arrow), NOT equals sign.

Selection: IF/ELSE IF/ELSE/ENDIF or CASE OF/ENDCASE.

Iteration: FOR (count-controlled), WHILE (pre-condition), REPEAT...UNTIL (post-condition).

Arrays: fixed size, same type, zero-indexed. Use loops to process arrays.

Procedures: no return. Functions: always return a value using RETURN.

String functions: LENGTH, LEFT, RIGHT, MID, UCASE, LCASE, &.

File handling: OPENFILE, READFILE/WRITEFILE, CLOSEFILE — always close files!

Examiner's Tip

IGCSE exam command words: 'Write pseudocode' = use Cambridge syntax exactly. 'Complete the trace table' = fill every cell, show all iterations. 'Identify an error' = state the line and explain the bug. 'Suggest an improvement' = be specific — vague answers score zero!